



Universidad Peruana de Ciencias Aplicadas

Ingeniería de Software

Periodo: 202610

1ASI0732 | Diseño de Experimentos de Ingeniería de Software

NRC: 16879

Docente: Alex Humberto S  nchez Ponce

Informe del Trabajo Final

Startup: Stoq

Producto: StockWise

Integrantes

U   Luciana Carolina Choquehuanca Nu  ez

U   Ronald Joel Peralta Chipa

U202210973 U   Sanchez Rios, Camila Cristina

U   Fabiola Del Rocio Salda  a Ayala

U  

Abril, 2026

Registro de Versiones del Informe

Version	Fecha	Autor	Descripcion de modificaciones
---------	-------	-------	-------------------------------

Project Report Collaboration Insights

Enlace del repositorio del informe

Enlace de los repositorios de la organizacion

Contenido

Tabla de contenidos

- [Registro de Versiones del Informe](#)
- [Project Report Collaboration Insights](#)
 - [Enlace del repositorio del informe](#)
 - [Enlace de los repositorios de la organizacion](#)
- [Contenido](#)
 - [Tabla de contenidos](#)
- [Student Outcome](#)
- [Part I: As-Is Software Project](#)
- [Capitulo I: Introduccion](#)
 - [1.1. Startup Profile](#)
 - [1.1.1. Descripcion de la Startup](#)
 - [1.1.2. Perfiles de integrantes del equipo](#)
 - [1.2. Solution Profile](#)
 - [1.2.1. Antecedentes y problematica](#)
 - [1.2.2. Lean UX Process](#)
 - [1.2.2.1. Lean UX Problem Statements](#)
 - [1.2.2.2. Lean UX Assumptions](#)
 - [1.2.2.3. Lean UX Hypothesis Statements](#)
 - [1.2.2.4. Lean UX Canvas](#)
 - [1.3. Segmentos objetivo](#)
- [Capitulo II: Requirements Elicitation & Analysis](#)
 - [2.1. Competidores](#)
 - [2.1.1. Analisis competitivo](#)
 - [2.1.2. Estrategias y tacticas frente a competidores](#)
 - [2.2. Entrevistas](#)
 - [2.2.1. Diseno de entrevistas](#)
 - [2.2.2. Registro de entrevistas](#)
 - [2.2.3. Analisis de entrevistas](#)
 - [2.3. Needfinding](#)
 - [2.3.1. User Personas](#)
 - [2.3.2. User Task Matrix](#)
 - [2.3.3. User Journey Mapping](#)
 - [2.3.4. Empathy Mapping](#)
 - [2.3.5. As-is Scenario Mapping](#)
 - [2.4. Ubiquitous Language](#)
- [Capitulo III: Requirements Specification](#)
 - [3.1. To-Be Scenario Mapping](#)
 - [3.2. User Stories](#)
 - [3.3. Product Backlog](#)
 - [3.4. Impact Mapping](#)
- [Capitulo IV: Product Design](#)

- 4.1. Style Guidelines
 - 4.1.1. General Style Guidelines
 - 4.1.2. Web Style Guidelines
 - 4.1.3. Mobile Style Guidelines
 - 4.1.3.1. iOS Mobile Style Guidelines
 - 4.1.3.2. Android Mobile Style Guidelines
- 4.2. Information Architecture
 - 4.2.1. Organization Systems
 - 4.2.2. Labeling Systems
 - 4.2.3. SEO Tags and Meta Tags
 - 4.2.4. Searching Systems
 - 4.2.5. Navigation Systems
- 4.3. Landing Page UI Design
 - 4.3.1. Landing Page Wireframe
 - 4.3.2. Landing Page Mock-up
- 4.4. Mobile Applications UX/UI Design
 - 4.4.1. Mobile Applications Wireframes
 - 4.4.2. Mobile Applications Wireflow Diagrams
 - 4.4.3. Mobile Applications Mock-ups
 - 4.4.4. Mobile Applications User Flow Diagrams
- 4.5. Mobile Applications Prototyping
 - 4.5.1. Android Mobile Applications Prototyping
 - 4.5.2. iOS Mobile Applications Prototyping
- 4.6. Web Applications UX/UI Design
 - 4.6.1. Web Applications Wireframes
 - 4.6.2. Web Applications Wireflow Diagrams
 - 4.6.3. Web Applications Mock-ups
 - 4.6.4. Web Applications User Flow Diagrams
- 4.7. Web Applications Prototyping
- 4.8. Domain-Driven Software Architecture
 - 4.8.1. Software Architecture Context Diagram
 - 4.8.2. Software Architecture Container Diagrams
 - 4.8.3. Software Architecture Components Diagrams
- 4.9. Software Object-Oriented Design
 - 4.9.1. Class Diagrams
 - 4.9.2. Class Dictionary
- 4.10. Database Design
 - 4.10.1. Relational/Non-Relational Database Diagram
- Capitulo V: Product Implementation
 - 5.1. Software Configuration Management
 - 5.1.1. Software Development Environment Configuration
 - 5.1.2. Source Code Management
 - 5.1.3. Source Code Style Guide & Conventions
 - 5.1.4. Software Deployment Configuration
 - 5.2. Product Implementation & Deployment
 - 5.2.1. Sprint Backlogs

- 5.2.2. Implemented Landing Page Evidence
- 5.2.3. Implemented Frontend-Web Application Evidence
- 5.2.4. Acuerdo de Servicio - SaaS
- 5.2.5. Implemented Native-Mobile Application Evidence
- 5.2.6. Implemented RESTful API and/or Serverless Backend Evidence
- 5.2.7. RESTful API documentation
- 5.2.8. Team Collaboration Insights
- 5.3. Video About-the-Product
- Part II: Verification, Validation & Pipeline
- Capitulo VI: Product Verification & Validation
 - 6.1. Testing Suites & Validation
 - 6.1.1. Core Entities Unit Tests
 - 6.1.2. Core Integration Tests
 - Información General Test-1
 - Escenario 1
 - Escenario 2
 - Escenario 3
 - Escenario 4
 - Información General Test-2
 - Escenario 1
 - Escenario 2
 - Escenario 3
 - Información General Test-3
 - Escenario 1
 - Escenario 2
 - Escenario 3
 - Información General Test-4
 - Escenario 1
 - Escenario 2
 - Información General Test-5
 - Escenario 1
 - Escenario 2
 - Escenario 3
 - 6.1.3. Core Behavior-Driven Development
 - Evidence 1: Product Registration Feature File
 - Evidence 2: Low Stock Alert Feature File
 - Evidence 3: BDD Test Execution Results
 - 6.1.4. Core System Tests
 - 6.2. Static testing & Verification
 - 6.2.1. Static Code Analysis
 - 6.2.1.1. .Coding standard & Code conventions
 - 6.2.1.2. Code Quality & Code Security.
 - 6.2.2. Reviews
 - 6.3. Validation Interviews.
 - 6.3.1. Diseño de Entrevistas.
 - 6.3.2. Registro de Entrevistas.

- 6.3.3. Evaluaciones segun heurísticas.
- 6.4. Auditoria de Experiencias de Usuario.
 - 6.4.1. Auditoria realizada.
 - 6.4.1.1. Información del grupo auditado.
 - 6.4.1.2. Cronograma de auditoria realizada.
 - 6.4.1.3. Contenido de auditoría realizada.
 - 6.4.2. Auditoria recibida.
 - 6.4.2.1. Información del grupo auditor.
 - 6.4.2.2. Cronograma de auditoría recibida.
 - 6.4.2.3. Contenido de auditoría recibida.
 - 6.4.2.4. Resumen de modificaciones para subsanar hallazgos.
- Capitulo VII: DevOps Practices
 - 7.1. Continuous Integration
 - 7.1.1. Tools and Practices
 - 7.1.2. Build & Test Suite Pipeline Components
 - 7.2. Continuous Delivery
 - 7.2.1. Tools and Practices
 - 7.2.2. Stages Deployment Pipeline Components
 - 7.3. Continuous deployment
 - 7.3.1. Tools and Practices
 - 7.3.2. Production Deployment Pipeline Components
 - 7.4. Continuous Monitoring
 - 7.4.1. Tools and Practices
 - 7.4.2. Monitoring Pipeline Components
 - 7.4.3. Alerting Pipeline Components
 - 7.4.4. Notification Pipeline Components
- Part III: Experiment-Driven Lifecycle
- Capitulo VIII: Experiment-Driven Development
 - 8.1. Experiment Planning
 - 8.1.1. As-Is Summary
 - 8.1.2. Raw Material: Assumptions, Knowledge Gaps, Ideas, Claims
 - 8.1.3. Experiment-Ready Questions
 - 8.1.4. Question Backlog
 - 8.1.5. Experiment Cards
 - 8.2. Experiment Design
 - 8.2.1. Hypotheses
 - 8.2.2. Domain Business Metrics
 - 8.2.3. Measures
 - 8.2.4. Conditions
 - 8.2.5. Scale Calculations and Decisions
 - 8.2.6. Methods Selection
 - 8.2.7. Data Analytics: Goals, KPIs and Metrics Selection
 - 8.2.8. Web and Mobile Tracking Plan
 - 8.3. Experimentation
 - 8.3.1. To-Be User Stories
 - 8.3.2. To-Be Product Backlog

- 8.3.3. Pipeline-supported, Experiment-Driven To-Be Software Platform Lifecycle
 - 8.3.3.1. To-Be Sprint Backlogs
 - 8.3.3.2. Implemented To-Be Landing Page Evidence
- 5.2.3. Implemented Frontend-Web Application Evidence
 - 8.3.3.4. Implemented To-Be Native-Mobile Application Evidence
 - 8.3.3.5. Implemented To-Be RESTful API and/or Serverless Backend Evidence
 - 8.3.3.6. Team Collaboration Insights
- 8.3.4. To-Be Validation Interviews
 - 8.3.4.1. Diseno de Entrevistas
 - 8.3.4.2. Registro de Entrevistas
- 8.4. Experiment Aftermath & Analysis
 - 8.4.1. Analysis and Interpretation of Results
 - 8.4.2. Re-scored and Re-prioritized Question Backlog
- 8.5. Continuous Learning
 - 8.5.1. Shareback Session Artifacts: Learning Workflow
- 8.6. To-Be Software Platform Pre-launch
 - 8.6.1. About-the-Product Intro Video
- Conclusiones
- Conclusiones y recomendaciones
- Bibliografia
- Anexos

Student Outcome

ABET – EAC - Student Outcome 4

Criterio: La capacidad de reconocer responsabilidades éticas y profesionales en situaciones de ingeniería y hacer juicios informados, que deben considerar el impacto de las soluciones de ingeniería en contextos globales, económicos, ambientales y sociales.

En el siguiente cuadro se describe las acciones realizadas y enunciados de conclusiones por parte del grupo, que permiten sustentar el haber alcanzado el logro del ABET – EAC - Student Outcome 4.

Criterio específico	Acciones realizadas	Conclusiones
Reconoce responsabilidad ética y profesional en situaciones de ingeniería de software	Miranda Ayasta, Rogger Faryd	
	AV1:	
	TB1:	
	AV2:	
	TB2:	
	Vargas Javier, Jose Enrique	
	AV1:	
	TB1:	
	AV2:	
	TB2:	
	Sanchez Rios, Camila	
	AV1:	
	TB1:	
	AV2:	
	TB2:	
	Luciana Carolina Choquehuanca Nuñez	
	AV1:	
	TB1:	

AV2:

TB2:

**Ronald Joel Peralta
Chipa**

AV1:

TB1:

AV2:

TB2:

**Emite juicios informados considerando el impacto de las
soluciones de ingeniería de software en contextos
globales, económicos, ambientales y sociales**

**Miranda Ayasta,
Rogger Faryd**

AV1:

TB1:

AV2:

TB2:

**Vargas Javier, Jose
Enrique**

AV1:

TB1:

AV2:

TB2:

Sanchez Rios, Camila

AV1:

TB1:

AV2:

TB2:

**Luciana Carolina
Choquehuanca Nuñez**

AV1:

TB1:

AV2:

TB2:

**Ronald Joel Peralta
Chipa**

AV1:

TB1:

AV2:

TB2:

Part I: As-Is Software Project

Capitulo I: Introduccion

1.1. Startup Profile

1.1.1. Descripcion de la Startup

1.1.2. Perfiles de integrantes del equipo

1.2. Solution Profile

1.2.1. Antecedentes y problematica

1.2.2. Lean UX Process

1.2.2.1. Lean UX Problem Statements

1.2.2.2. Lean UX Assumptions

1.2.2.3. Lean UX Hypothesis Statements

1.2.2.4. Lean UX Canvas

1.3. Segmentos objetivo

Capitulo II: Requirements Elicitation & Analysis

2.1. Competidores

2.1.1. Analisis competitivo

2.1.2. Estrategias y tacticas frente a competidores

2.2. Entrevistas

2.2.1. Diseno de entrevistas

2.2.2. Registro de entrevistas

2.2.3. Analisis de entrevistas

2.3. Needfinding

2.3.1. User Personas

2.3.2. User Task Matrix

2.3.3. User Journey Mapping

2.3.4. Empathy Mapping

2.3.5. As-is Scenario Mapping

2.4. Ubiquitous Language

Capitulo III: Requirements Specification

3.1. To-Be Scenario Mapping

3.2. User Stories

3.3. Product Backlog

3.4. Impact Mapping

Capitulo IV: Product Design

4.1. Style Guidelines

4.1.1. General Style Guidelines

4.1.2. Web Style Guidelines

4.1.3. Mobile Style Guidelines

4.1.3.1. iOS Mobile Style Guidelines

4.1.3.2. Android Mobile Style Guidelines

4.2. Information Architecture

4.2.1. Organization Systems

4.2.2. Labeling Systems

4.2.3. SEO Tags and Meta Tags

4.2.4. Searching Systems

4.2.5. Navigation Systems

4.3. Landing Page UI Design

4.3.1. Landing Page Wireframe

4.3.2. Landing Page Mock-up

4.4. Mobile Applications UX/UI Design

4.4.1. Mobile Applications Wireframes

4.4.2. Mobile Applications Wireflow Diagrams

4.4.3. Mobile Applications Mock-ups

4.4.4. Mobile Applications User Flow Diagrams

4.5. Mobile Applications Prototyping

4.5.1. Android Mobile Applications Prototyping

4.5.2. iOS Mobile Applications Prototyping

4.6. Web Applications UX/UI Design

4.6.1. Web Applications Wireframes

4.6.2. Web Applications Wireflow Diagrams

4.6.3. Web Applications Mock-ups

4.6.4. Web Applications User Flow Diagrams

4.7. Web Applications Prototyping

4.8. Domain-Driven Software Architecture

4.8.1. Software Architecture Context Diagram

4.8.2. Software Architecture Container Diagrams

4.8.3. Software Architecture Components Diagrams

4.9. Software Object-Oriented Design

4.9.1. Class Diagrams

4.9.2. Class Dictionary

4.10. Database Design

4.10.1. Relational/Non-Relational Database Diagram

Capitulo V: Product Implementation

5.1. Software Configuration Management

5.1.1. Software Development Environment Configuration

5.1.2. Source Code Management

5.1.3. Source Code Style Guide & Conventions

5.1.4. Software Deployment Configuration

5.2. Product Implementation & Deployment

5.2.1. Sprint Backlogs

5.2.2. Implemented Landing Page Evidence

5.2.3. Implemented Frontend-Web Application Evidence

5.2.4. Acuerdo de Servicio - SaaS

5.2.5. Implemented Native-Mobile Application Evidence

5.2.6. Implemented RESTful API and/or Serverless Backend Evidence

5.2.7. RESTful API documentation

5.2.8. Team Collaboration Insights

5.3. Video About-the-Product

Part II: Verification, Validation & Pipeline

Capítulo VI: Product Verification & Validation

6.1. Testing Suites & Validation

6.1.1. Core Entities Unit Tests

Se muestra evidencia de los test a las historias de usuario del proyecto

US01 - Crear producto válido: valida que el agregado Product se construye correctamente con datos completos y normaliza los valores de texto.

```
@Test
// US01 - Registrar producto nuevo: creación válida.
void shouldCreateProductWithValidData() {
    CreateProductCommand command = new CreateProductCommand(
        name: " Arroz Extra ",
        description: " Bolsa de 5kg ",
        categoryId: "1",
        providerId: "10",
        minStock: 8,
        unitPrice: 12.5,
        isActive: true
    );

    Product product = new Product(command);

    assertNotNull(product);
    assertEquals(expected: "Arroz Extra", product.getName());
    assertEquals(expected: "Bolsa de 5kg", product.getDescription());
    assertEquals(expected: "1", product.getCategoryId());
    assertEquals(expected: "10", product.getProviderId());
    assertEquals(expected: 8, product.getMinStock());
    assertEquals(expected: 12.5, product.getUnitPrice());
    assertEquals(expected: true, product.getIsActive());
}
```

US01 / US19 - Stock inicial y mínimo: valida que minStock acepta valores válidos, incluyendo cero, y rechaza negativos.


```
@Test
// US01 + US19 - stock inicial/minStock válido en 0.
void shouldAllowZeroMinStockAsInitialStockThreshold() {
    CreateProductCommand command = new CreateProductCommand(
        name: "Aceite",
        description: "Botella 1l",
        categoryId: "2",
        providerId: "11",
        minStock: 0,
        unitPrice: 7.2,
        isActive: true
    );

    Product product = new Product(command);
    assertEquals(expected: 0, product.getMinStock());
}
```

US19 - Configurar stock mínimo: valida que el producto permita actualizar el límite mínimo de stock correctamente.

```
@Test
// US19 - Configurar stock mínimo: guardar nuevo límite.
void shouldUpdateMinStockWhenCommandIsValid() {
    Product product = new Product(new CreateProductCommand(
        name: "Fideos",
        description: "Paquete",
        categoryId: "3",
        providerId: "12",
        minStock: 4,
        unitPrice: 3.5,
        isActive: true
    ));

    UpdateProductCommand update = new UpdateProductCommand(
        productId: 1L,
        name: "Fideos Premium",
        description: "Paquete grande",
        categoryId: "3",
        providerId: "12",
        minStock: 9,
        unitPrice: 4.0,
        isActive: true
    );

    product.updateProduct(update);

    assertEquals(expected: "Fideos Premium", product.getName());
    assertEquals(expected: 9, product.getMinStock());
    assertEquals(expected: 4.0, product.getUnitPrice());
}
```

US03 - Registrar salida de producto: valida que Batch descuenta stock correctamente al registrar una salida.

```
@Test
// US03 - Registrar salida de producto: estado base para operar stock.
void shouldCreateBatchWithValidData() {
    Date now = new Date();
    Date expiration = new Date(now.getTime() + 86_400_000L);

    Batch batch = new Batch(new CreateBatchCommand(productId: 1L, quantity: 20, expiration, now));

    assertEquals(expected: 1L, batch.getProductId());
    assertEquals(expected: 20, batch.getQuantity());
}

@Test
// US03 - Registrar salida de producto: descuento correcto del stock.
void shouldDecreaseStockForProductOutput() {
    Date now = new Date();
    Date expiration = new Date(now.getTime() + 86_400_000L);
    Batch batch = new Batch(new CreateBatchCommand(productId: 5L, quantity: 15, expiration, now));

    batch.reduceQuantity(amount: 6);

    assertEquals(expected: 9, batch.getQuantity());
}
```

US14 - Registrar devolución: valida que el stock aumente al registrar una devolución.

```
@Test
// US14 - Registrar devolución: aumentar stock.
void shouldIncreaseStockWhenRegisteringReturn() {
    Date now = new Date();
    Date expiration = new Date(now.getTime() + 86_400_000L);
    Batch batch = new Batch(new CreateBatchCommand(productId: 7L, quantity: 10, expiration, now));

    int returnedQuantity = 3;
    batch.setQuantity(batch.getQuantity() + returnedQuantity);

    assertEquals(expected: 13, batch.getQuantity());
}
```

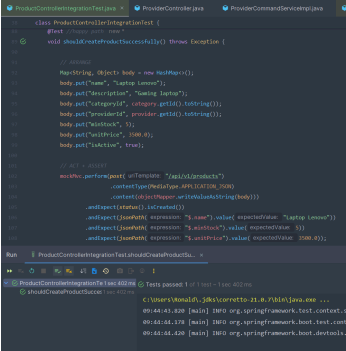
6.1.2. Core Integration Tests

En esta sección se presentan las pruebas de integración desarrolladas para validar la correcta comunicación e interacción entre los componentes principales de StockWise, garantizando el funcionamiento coordinado y consistente de los distintos módulos que conforman el sistema.

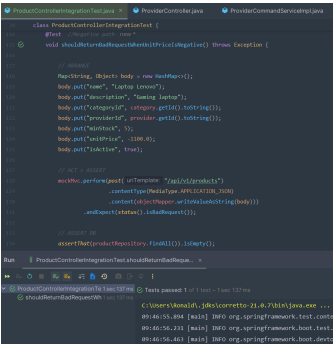
Información General Test-1

Elemento	Descripción
Clase de Test	ProductControllerIntegrationTest
Módulo(s)	inventory, shared, iam - US12, US13
Descripción General	Validar la integración completa del módulo de productos dentro del sistema StockTrack, comprobando que las operaciones relacionadas con la gestión de productos funcionen correctamente desde la capa HTTP hasta la persistencia en base de datos.

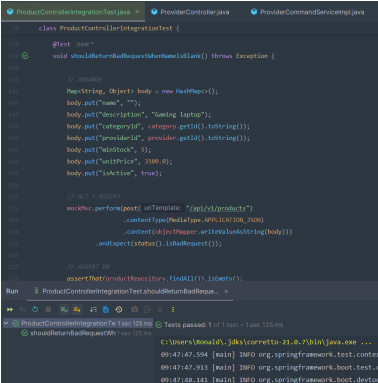
Escenario 1

Evidencia	Descripción
	Este escenario valida que el sistema pueda registrar correctamente un nuevo producto cuando se envían datos válidos, verificando el flujo completo desde la solicitud HTTP hasta la persistencia en base de datos. Se comprueba la integración entre controller, servicios de aplicación, aggregate Product, repositorios JPA y la base de datos H2. Esta validación es importante porque asegura que la funcionalidad principal de creación de productos opere correctamente y que la información registrada mantenga consistencia dentro del inventario.

Escenario 2

Evidencia	Descripción
	Este escenario valida que el sistema rechace productos con un precio unitario inválido, específicamente valores negativos, comprobando que las reglas de negocio definidas en el dominio sean respetadas antes de persistir información. La prueba garantiza que las validaciones del aggregate y el manejo de excepciones funcionen correctamente a través de toda la integración del sistema. Esto es importante para proteger la integridad de los datos financieros y evitar inconsistencias en cálculos de ventas, reportes y valorización de inventario.

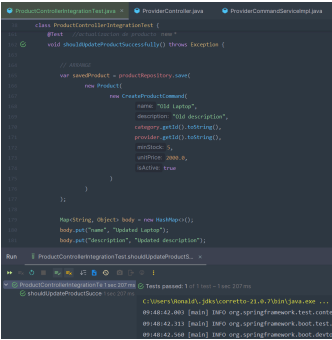
Escenario 3

Evidencia	Descripción
	Este escenario verifica que el sistema no permita registrar productos sin nombre o con nombres vacíos, validando las restricciones obligatorias del dominio y el correcto manejo de respuestas HTTP ante entradas inválidas. La prueba asegura que los datos esenciales del catálogo sean obligatorios y que el sistema prevenga registros incompletos que puedan afectar búsquedas, clasificación de productos y operaciones posteriores dentro del inventario.

Escenario 4

Evidencia

Descripción



Este escenario valida que el sistema pueda actualizar correctamente la información de un producto existente, verificando la integración entre el endpoint REST, los servicios de actualización, el aggregate Product, el repositorio JPA y la persistencia en base de datos. Se comprueba que los cambios enviados sean reflejados correctamente y que las validaciones del dominio continúen aplicándose durante la edición. Esta validación es importante porque garantiza que la información del catálogo pueda mantenerse actualizada y consistente a lo largo del tiempo.

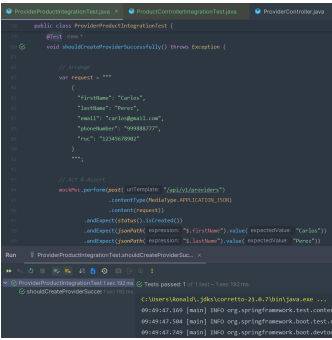
Información General Test-2

Elemento	Descripción
Clase de Test	ProviderProductIntegrationTest
Módulo(s)	inventory, shared - US24, US26
Descripción General	Validar la integración entre los módulos de proveedores y productos, asegurando que las relaciones funcionales entre ambos aggregates se comporten correctamente dentro del sistema.

Escenario 1

Evidencia

Descripción

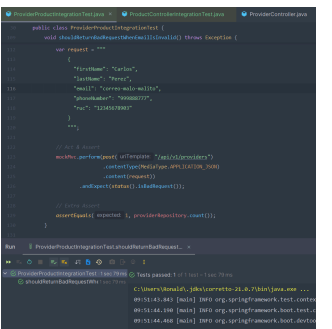


Este escenario valida que el sistema pueda registrar correctamente un nuevo proveedor utilizando datos válidos, comprobando el flujo completo desde la solicitud HTTP hasta la persistencia en base de datos. La prueba verifica la integración entre controller, servicios de aplicación, aggregate Provider, value objects, repositorios JPA y la base de datos H2. Esta validación es importante porque garantiza que los proveedores puedan ser gestionados correctamente y que la información crítica de abastecimiento quede almacenada de manera consistente y segura.

Escenario 2

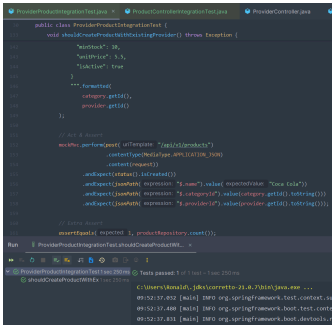
Evidencia

Descripción



Este escenario valida que el sistema rechace proveedores con correos electrónicos inválidos, comprobando que las reglas de validación implementadas en los value objects del dominio funcionen correctamente durante todo el flujo de integración. La prueba asegura que las excepciones generadas por datos inconsistentes sean manejadas adecuadamente y que la información inválida no llegue a persistirse en la base de datos. Esta validación es importante porque protege la calidad de los datos y evita problemas posteriores relacionados con notificaciones, contacto con proveedores y trazabilidad del sistema.

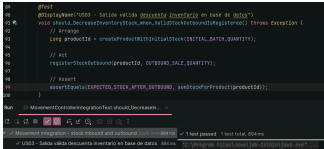
Escenario 3

Evidencia	Descripción
	Este escenario valida la integración funcional entre los módulos de productos y proveedores, verificando que un producto solo pueda registrarse cuando se encuentra asociado a un proveedor existente dentro del sistema. La prueba comprueba la interacción entre múltiples aggregates, servicios de aplicación, repositorios y validaciones de negocio relacionadas con integridad referencial. Esta validación es importante porque asegura consistencia entre entidades del inventario y evita relaciones inválidas que puedan afectar procesos de compras, abastecimiento y trazabilidad de productos.

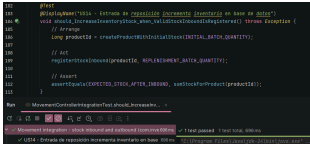
Información General Test-3

Elemento	Descripción
Clase de Test	MovementControllerIntegrationTest
Módulo(s)	inventory, sales - US03, US14
Descripción General	Validar la integración entre los módulos de inventario y ventas, comprobando que los movimientos de entrada y salida de stock actualicen correctamente la cantidad disponible en base de datos, y que el sistema rechace operaciones que excedan el stock disponible.

Escenario 1

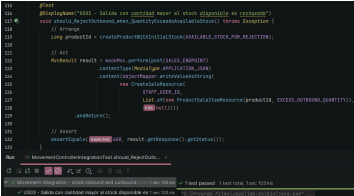
Evidencia	Descripción
	Este escenario valida que el sistema descuente correctamente el stock de un producto cuando se registra una venta válida, verificando el flujo completo desde la solicitud HTTP hasta la persistencia en base de datos H2. Se comprueba la integración entre el controlador de ventas, los servicios de aplicación, el repositorio de lotes y la lógica de descuento del aggregate. Esta validación es importante porque garantiza que cada transacción de salida mantenga la consistencia del inventario y refleje el stock real disponible para operaciones posteriores.

Escenario 2

Evidencia	Descripción
	Este escenario valida que el sistema incremente correctamente el stock de un producto cuando se registra un lote de reposición, comprobando la integración entre el endpoint de lotes, los servicios de aplicación, el repositorio JPA y la base de datos H2. La prueba verifica que la cantidad del nuevo lote se sume correctamente al stock existente del producto. Esta validación es importante porque asegura que el proceso de reabastecimiento opere de

Evidencia	Descripción
	forma confiable y que el inventario refleje en todo momento las entradas registradas por el equipo de almacén.

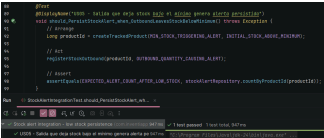
Escenario 3

Evidencia	Descripción
	Este escenario verifica que el sistema rechace una solicitud de venta cuya cantidad supera el stock disponible del producto, comprobando que las reglas de negocio del dominio sean aplicadas correctamente antes de persistir cualquier cambio. La prueba garantiza que el sistema retorne un error HTTP 400 ante este tipo de operación inválida. Esta validación es importante porque protege la integridad del inventario y evita que el sistema registre ventas que no pueden ser satisfechas con el stock existente.

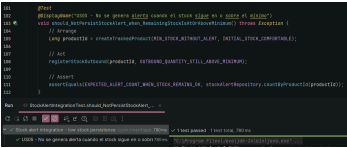
Información General Test-4

Elemento	Descripción
Clase de Test	StockAlertIntegrationTest
Módulo(s)	inventory, alertstock - US05
Descripción General	Validar la integración entre los módulos de inventario y alertas de stock, comprobando que el sistema genere y persista automáticamente una alerta cuando una salida de stock deja el nivel por debajo del mínimo configurado, y que no genere alertas innecesarias cuando el stock se mantiene dentro del rango aceptable.

Escenario 1

Evidencia	Descripción
	Este escenario valida que el sistema genere y persista automáticamente una alerta de stock bajo cuando una venta reduce el inventario por debajo del mínimo configurado para el producto, comprobando la integración entre el módulo de ventas, el servicio de alertas y el repositorio de alertas en base de datos H2. Esta validación es importante porque garantiza que el mecanismo de alerta temprana funcione de forma automática y confiable, permitiendo al equipo de almacén reaccionar oportunamente ante situaciones de desabastecimiento.

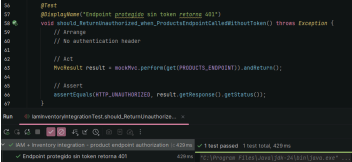
Escenario 2

Evidencia	Descripción
	Este escenario verifica que el sistema no genere alertas de stock cuando una salida de inventario deja el nivel igual o por encima del mínimo configurado, comprobando que la lógica de disparo de alertas sea precisa y no produzca falsos positivos. La prueba consulta directamente el repositorio de alertas para confirmar que ningún registro fue persistido. Esta validación es importante porque evita que el equipo de almacén reciba notificaciones innecesarias que puedan generar ruido operativo y reducir la efectividad del sistema de alertas.

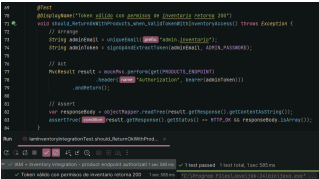
Información General Test-5

Elemento	Descripción
Clase de Test	IamInventoryIntegrationTest
Módulo(s)	iam, inventory - US01, US12
Descripción General	Validar la integración entre el módulo de autenticación y autorización (IAM) y los endpoints de inventario, comprobando que el sistema proteja correctamente el acceso a los recursos según el token y los permisos del usuario autenticado.

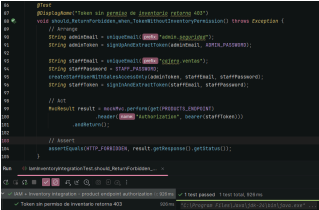
Escenario 1

Evidencia	Descripción
	Este escenario valida que el sistema rechace con HTTP 401 cualquier solicitud al endpoint de productos que no incluya un token de autenticación, comprobando que los filtros de seguridad estén activos y configurados correctamente. Esta validación es importante porque garantiza que los recursos del inventario estén protegidos frente a accesos anónimos y que el sistema cumpla con los requisitos de seguridad definidos para la plataforma SaaS.

Escenario 2

Evidencia	Descripción
	Este escenario valida que un usuario autenticado con permisos de inventario pueda acceder correctamente al endpoint de productos, recibiendo una respuesta HTTP 200 con el listado correspondiente. La prueba comprueba la integración completa entre el proceso de registro, la generación del token JWT y la autorización en el endpoint protegido. Esta validación es importante porque asegura que el flujo de autenticación funcione de extremo a extremo y que los usuarios con los permisos correctos puedan operar sin restricciones.

Escenario 3

Evidencia	Descripción
	Este escenario verifica que un usuario autenticado pero sin permisos de inventario reciba una respuesta HTTP 403 al intentar acceder al endpoint de productos, comprobando que el sistema aplique correctamente las reglas de autorización por rol. La prueba crea un usuario con acceso exclusivo a ventas e intenta acceder a un recurso restringido. Esta validación es importante porque garantiza el principio de mínimo privilegio en la plataforma, protegiendo los datos del inventario frente a accesos no autorizados de usuarios con roles insuficientes.

6.1.3. Core Behavior-Driven Development

En esta sección se presentan las pruebas Behavior-Driven Development desarrolladas para validar el comportamiento esperado de las funcionalidades principales de StockWise desde la perspectiva del usuario. El desarrollo de estas pruebas se basó en las User Stories priorizadas del Product Backlog, específicamente US01 - Registrar producto nuevo y US05 - Generar alertas por bajo stock, ambas pertenecientes al epic EP01 - Funciones básicas de inventario. Estas historias fueron seleccionadas porque representan funciones centrales del sistema de inventario.

Evidence 1: Product Registration Feature File

Feature: Registro de productos	Para mantener un control actualizado y confiable de las existencias desde su ingreso Como Usuario de inventario Quiero registrar un nuevo producto en mi inventario
Scenario: Registro exitoso de producto	Given el usuario proporciona información válida del producto When el usuario confirma el registro Then el sistema valida la información And registra los datos de manera segura And actualiza el inventario con el nuevo producto
Scenario: Información incompleta o inconsistente	Given existe información de producto incompleta o inconsistente When el sistema procesa la solicitud de registro Then rechaza la operación And genera una notificación indicando la necesidad de completar la información requerida
Scenario: Producto duplicado	Given existe una solicitud de registro con un producto ya registrado When el sistema procesa la información recibida Then impide el registro duplicado And genera una notificación indicando que el producto ya se encuentra en el inventario

Evidence 2: Low Stock Alert Feature File

```
Feature: Alertas por bajo stock

Para realizar el reabastecimiento oportuno y evitar quiebres de inventario
Como Usuario de inventario
Quiero recibir alertas cuando un producto se encuentra por debajo del stock mínimo

Scenario: Activación automática de alerta por bajo stock
    Given un producto tiene configurado un límite mínimo de stock
    When el sistema detecta que la cantidad disponible desciende por debajo de ese valor
    Then el sistema genera una alerta automática
    And comunica al usuario la necesidad de reabastecer el producto

Scenario: Personalización del límite de alerta
    Given el usuario desea modificar el valor mínimo de stock
    When el usuario establece un nuevo parámetro de alerta
    Then el sistema guarda el nuevo valor
    And aplica las alertas futuras con base en el límite configurado

Scenario: Desactivación temporal de alertas
    Given el usuario necesita suspender las notificaciones por mantenimiento o revisión de inventario
    When el usuario desactiva temporalmente las alertas
    Then el sistema detiene la emisión de nuevas alertas
    And registra la acción para su trazabilidad
```

Evidence 3: BDD Test Execution Results

```
Project ▾ pom.xml (stocktrack-backend) product-registration.feature low-stock-alert.feature CucumberTest.java x ProductRegistrationSteps.java
Run CucumberTest x
CucumberTest (com.inventiapp.stocktrack: 144ms) ✓ 6 tests passed 6 tests total, 144ms
  ✓ Alertas por bajo stock 117ms
    ✓ Activación automática de alerta por bajo stock 96ms
    ✓ Personalización del límite de alerta 11ms
    ✓ Desactivación temporal de alertas 10ms
  ✓ Registro de productos 27ms
    ✓ Registro exitoso de producto 9ms
    ✓ Información incompleta o inconsistente 8ms
    ✓ Producto duplicado 10ms

Scenario: Activación automática de alerta por bajo stock # features/low-stock-alert.feature:7
  Given un producto tiene configurado un límite mínimo de stock # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.unProductoTieneConfiguradoUnLi
  When el sistema detecta que la cantidad disponible desciende por debajo de ese valor # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaDetectaCantidadDispon
  Then el sistema genera una alerta automática # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaGeneraUnaAlertaAutoma
  And comunica al usuario la necesidad de reabastecer el producto # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.comunicaAlUsuarioLaNecesidadDe

Scenario: Personalización del límite de alerta # features/low-stock-alert.feature:13
  Given el usuario desea modificar el valor mínimo de stock # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioDeseaModificarElValorMinimoDeStock()
  When el usuario establece un nuevo parámetro de alerta # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioEstableceUnNuevoParametroDeAlerta()
  Then el sistema guarda el nuevo valor # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaGuardaElNuevoValor()
  And aplica las alertas futuras con base en el límite configurado # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.aplicaLasAlertasFuturasConBaseEnElLimiteConfigurad

Scenario: Desactivación temporal de alertas # features/low-stock-alert.feature:19
  Given el usuario necesita suspender las notificaciones por mantenimiento o revisión de inventario # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioNecesita
  When el usuario desactiva temporalmente las alertas # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioDesactiv
  Then el sistema detiene la emisión de nuevas alertas # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaDetieneE
  And registra la acción para su trazabilidad # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.registraLaAccionP

Scenario: Registro exitoso de producto # features/product-registration.feature:7
  Given el usuario proporciona información válida del producto # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioProporcionaInformacionValidaDelProducto()
  When el usuario confirma el registro # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elUsuarioConfirmaElRegistro()
  Then el sistema valida la información # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaValidaLaInformacion()
  And registra los datos de manera segura # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.registraLosDatosDeManeraSegura()
  And actualiza el inventario con el nuevo producto # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.actualizaElInventarioConELNuevoProducto()

Scenario: Información incompleta o inconsistente # features/product-registration.feature:14
  Given existe información de producto incompleta o inconsistente # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.existeInformacionDeProduct
  When el sistema procesa la solicitud de registro # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaProcesaLaSolicitu
  Then rechaza la operación # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.rechazaLaOperacion()
  And genera una notificación indicando la necesidad de completar la información requerida # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.generaNotificacionIndicand

Scenario: Producto duplicado # features/product-registration.feature:20
  Given existe una solicitud de registro con un producto ya registrado # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.existeUnaSolicitudDeRegistru
  When el sistema procesa la información recibida # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.elSistemaProcesaLaInformacio
  Then impide el registro duplicado # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.impideElRegistroDuplicado()
  And genera una notificación indicando que el producto ya se encuentra en el inventario # com.inventiapp.stocktrack.bdd.ProductRegistrationSteps.generaNotificacionProductoYa
```

6.1.4. Core System Tests

Feature: Confirmar salida y descontar inventario

Para mantener la exactitud del stock y asegurar el descuento correcto de los lotes

Como Cajero

Quiero confirmar una salida de productos en el sistema

Scenario: Venta básica con consumo secuencial de lotes

Given el sistema cuenta con un producto activo con stock distribuido en múltiples lotes

When el usuario procesa una venta por una cantidad que supera el primer lote pero es cubierta por el segundo

Then el sistema valida y procesa la solicitud exitosamente

And agota por completo el inventario del primer lote

And descuenta la cantidad restante del segundo lote

And registra la venta de manera segura

```
@Test
@DisplayName("Debe completar flujo de venta basico")
void testSimpleSalesFlow() throws Exception {

    categoryId = extractIdFromResponse(postRequest( endpoint: "/api/v1/categories",
        new CreateCategoryResource( name: "Electronics", description: "Electronics products")));

    providerId = extractIdFromResponse(postRequest( endpoint: "/api/v1/providers",
        new CreateProviderResource( firstName: "Miguel", lastName: "Suarez", phoneNumber: "1234567890", email: "tech@email.com", ruc: "20123456789")));

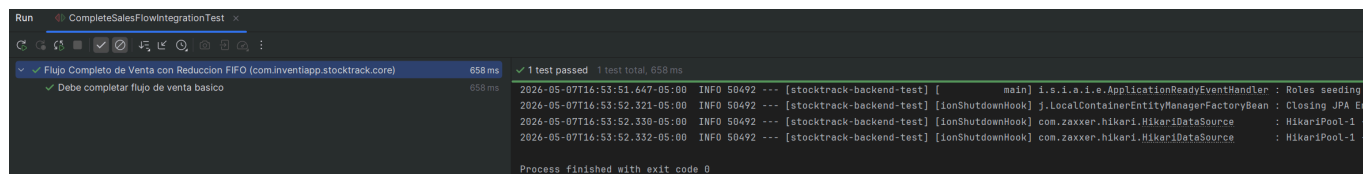
    productId = extractIdFromResponse(postRequest( endpoint: "/api/v1/products",
        new CreateProductResource( name: "USB Keyboard", description: "Mechanical RGB Keyboard",
            String.valueOf(categoryId), String.valueOf(providerId), minStock: 5, unitPrice: 29.99, isActive: true)));

    Date now = new Date();
    Long batch1Id = extractIdFromResponse(postRequest( endpoint: "/api/v1/batches",
        new CreateBatchResource(productId, quantity: 50, addDays(25), now)));
    Long batch2Id = extractIdFromResponse(postRequest( endpoint: "/api/v1/batches",
        new CreateBatchResource(productId, quantity: 30, addDays(40), now)));

    postRequest( endpoint: "/api/v1/sales", new CreateSaleResource( staffUserId: 1L, List.of(new ProductSaleItemResource(productId, quantity: 60)), kits: null));

    // Se valida solo el resultado clave del flujo
    getAndValidate( endpoint: "/api/v1/batches/" + batch1Id, expectedQuantity: 0);
    getAndValidate( endpoint: "/api/v1/batches/" + batch2Id, expectedQuantity: 20);

    mockMvc.perform(get( uriTemplate: "/api/v1/sales"))
        .andExpect(status().isOk())
        .andExpect(jsonPath( expression: "$", hasSize(greaterThanOrEqualTo( value: 1))));
}
```



6.2. Static testing & Verification

Esta sección describe las actividades de control de calidad que se ejecutan directamente sobre el código fuente, especificaciones y artefactos del proyecto sin necesidad de poner el sistema en marcha. Su objetivo es identificar de manera temprana ambigüedades, defectos de diseño, vulnerabilidades de seguridad y desviaciones de las pautas de desarrollo establecidas.

6.2.1. Static Code Analysis

Consiste en la evaluación automatizada del código base utilizando herramientas especializadas. A través de este análisis se audita la mantenibilidad, el cumplimiento de reglas sintácticas, la deuda técnica y los posibles riesgos de seguridad antes de que el código sea integrado a las ramas principales del repositorio.

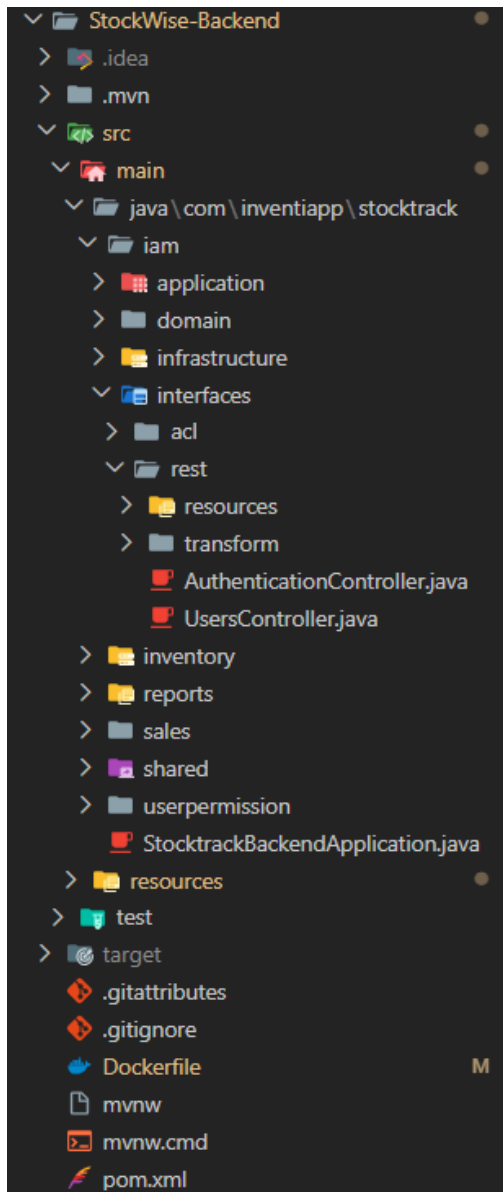
6.2.1.1. Coding standard & Code conventions

Backend - Java/Spring Boot

El backend implementa convenciones de código rigurosas basadas en los estándares oficiales de Spring Framework y las mejores prácticas de la arquitectura limpia. La nomenclatura sigue patrones semánticos y descriptivos: los controladores utilizan el sufijo Controller, los servicios emplean Service o la distinción CommandService / QueryService para segregar las operaciones de lectura y escritura (patrón CQRS), y los componentes de transformación adoptan el sufijo Assembler para la conversión eficiente entre entidades del dominio y recursos REST (DTOs).

La estructura de paquetes refleja fielmente una arquitectura hexagonal, donde cada contexto acotado (Bounded Context) está claramente delimitado en los paquetes domain, application, interfaces.rest e infrastructure, facilitando una estricta separación de responsabilidades. Adicionalmente, todos los métodos públicos incluyen documentación técnica mediante Javadoc, detallando parámetros y valores de retorno. El manejo de excepciones es homogéneo mediante una capa global de captura de errores, la cual procesa excepciones de negocio específicas (tales como ProductAlreadyExistsException o ProductNotFoundException) y las traduce en códigos de estado HTTP semánticos (201, 400, 404, 500).

Las anotaciones nativas de Spring (@RestController, @PostMapping, @GetMapping) y de documentación (@Operation, @ApiResponse) se aplican de forma transversal para generar de manera automática el contrato de la API mediante OpenAPI/Swagger. Finalmente, la inyección de dependencias se realiza exclusivamente a través de constructores, promoviendo la inmutabilidad de los componentes y simplificando el diseño de pruebas unitarias.



```

@RestController
@RequestMapping(value = "/api/v1/authentication", produces = MediaType.APPLICATION_JSON_VALUE)
@Tag(name = "Authentication", description = "Authentication Endpoints")
public class AuthenticationController {

    private final UserCommandService userCommandService;
    private final TokenService tokenService;

    /**
     * Constructor
     * @param userCommandService The {@link UserCommandService} instance
     * @param tokenService The {@link TokenService} instance
     */
    public AuthenticationController(UserCommandService userCommandService, TokenService tokenService) {
        this.userCommandService = userCommandService;
        this.tokenService = tokenService;
    }

    /**
     * Sign in endpoint
     * @param signInResource The {@link SignInResource} instance
     * @return A {@link AuthenticatedUserResource} resource for the authenticated user with JWT token, or an unauthorized response if credentials are invalid
     */
    @PostMapping("/sign-in")
    @Operation(summary = "Sign in", description = "Authenticate a user and return a JWT token")
    @ApiResponse(value = {
        @ApiResponse(responseCode = "200", description = "User authenticated successfully"),
        @ApiResponse(responseCode = "401", description = "Invalid credentials")
    })
    public ResponseEntity<AuthenticatedUserResource> signIn(@RequestBody SignInResource signInResource) {
        var signInCommand = SignInCommandFromResourceAssembler.toCommandFromResource(signInResource);
        var result = userCommandService.handle(signInCommand);

        if (result.isEmpty()) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
        }

        var user = result.get().getLeft();
        var token = result.get().getRight();
        var authenticatedUserResource = AuthenticatedUserResourceFromEntityAssembler.toResourceFromEntity(user, token);

        return ResponseEntity.ok(authenticatedUserResource);
    }
}

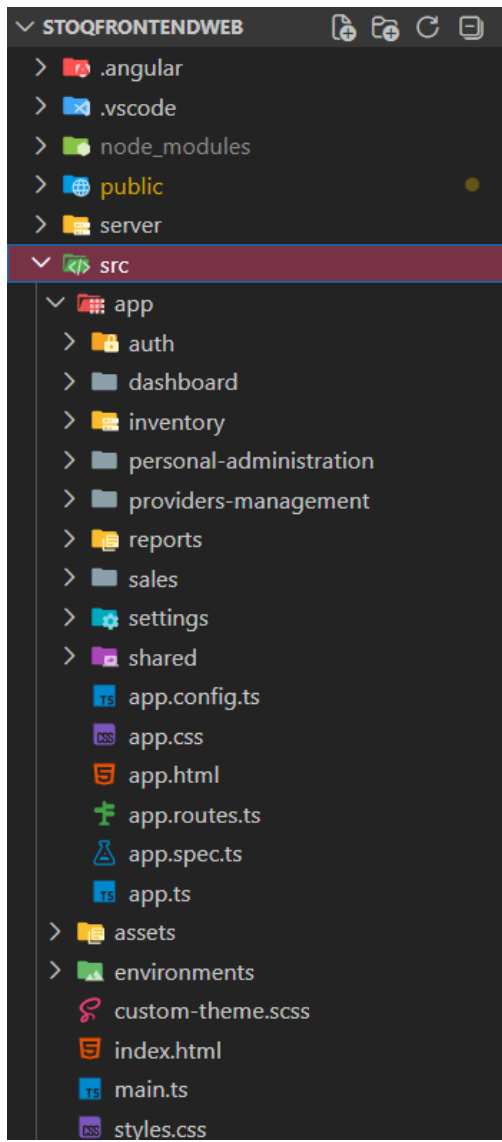
```

Frontend - Angular/TypeScript

El frontend se alinea con las convenciones oficiales recomendadas por el equipo de Angular y las directrices de TypeScript, estructurándose mediante una arquitectura modular orientada a características (Features). Cada módulo de negocio (auth, inventory, dashboard, etc.) replica la filosofía de separación de responsabilidades en las capas de presentation, domain, infrastructure y application.

Los archivos se nombran estrictamente utilizando la convención kebab-case (auth-api.ts, auth-guard.ts, inventory.store.ts), mientras que las clases e interfaces emplean PascalCase (AuthApi, InventoryStore, AuthGuard). Los servicios inyectables se configuran mediante el decorador @Injectable({ providedIn: 'root' }), lo que optimiza el empaquetado mediante tree-shaking y previene la duplicidad de instancias. Se destaca el uso de Angular Signals para la gestión del estado reactivo de la interfaz, adoptando así la API moderna del framework para reducir la complejidad operativa frente a patrones RxJS tradicionales.

La documentación mediante JSDoc es mandatoria en métodos críticos, especialmente en guards y servicios de dominio. Los guards de protección de rutas implementan la interfaz funcional CanActivateFn, validando la vigencia del token JWT, los estados de autenticación y los roles del usuario antes de permitir el acceso a las vistas. El código prohíbe la lógica compleja dentro de los templates HTML, delegando dicha responsabilidad a los componentes y almacenes (stores) fuertemente tipados.



```

export interface StockInfo {
  productId: string;
  currentStock: number;
  lastUpdated: string;
}

> /**...
@Injectable({
  providedIn: 'root'
})
export class InventoryStore {
  private readonly productsSignal = signal<Product[]>([]);
  private readonly categoriesSignal = signal<Category[]>([]);
  private readonly providersSignal = signal<Provider[]>([]);
  private readonly kitsSignal = signal<Kit[]>([]);
  private readonly batchesSignal = signal<Batch[]>([]);
  private readonly loadingSignal = signal<boolean>(false);
  private readonly errorSignal = signal<string | null>(null);

  readonly products = this.productsSignal.asReadonly();
  readonly categories = this.categoriesSignal.asReadonly();
  readonly providers = this.providersSignal.asReadonly();
  readonly kits = this.kitsSignal.asReadonly();
  readonly batches = this.batchesSignal.asReadonly();
  readonly loading = this.loadingSignal.asReadonly();
  readonly error = this.errorSignal.asReadonly();

  /**
   * Computed stock from batches - calculates total quantity per product.
   */
  readonly stock = computed<StockInfo[]>(() => {
    const batches = this.batches();
    const stockMap = new Map<string, { quantity: number; lastDate: string }>();

    batches.forEach(batch => {
      const productId = String(batch.productId);
      const existing = stockMap.get(productId);
      const batchDate = batch.receptionDate || new Date().toISOString();

      if (existing) {
        existing.quantity += batch.quantity;
        if (batchDate > existing.lastDate) {
          existing.lastDate = batchDate;
        }
      }
    });
  });
}

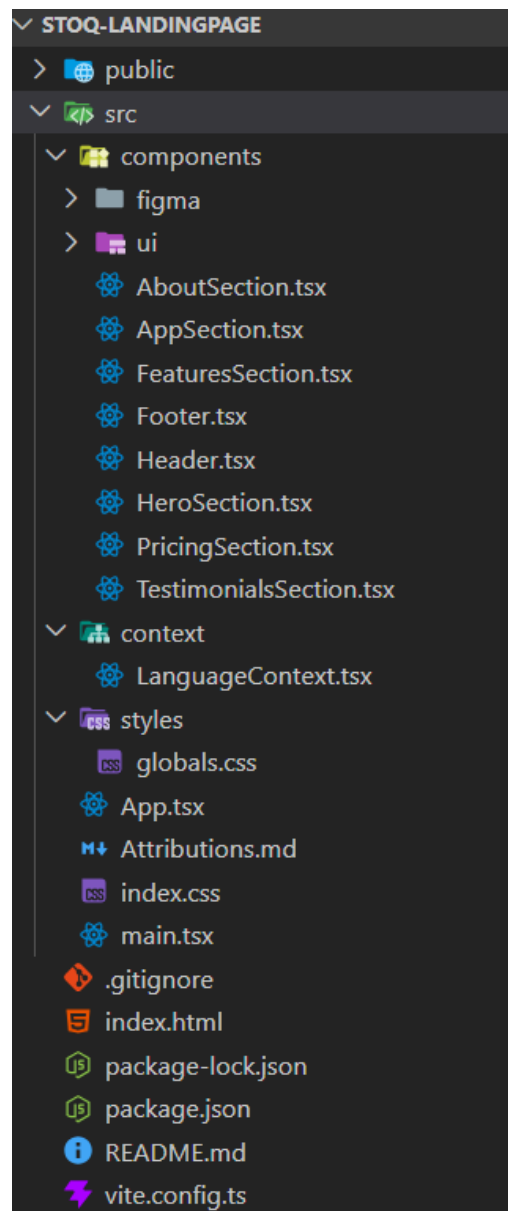
```

Landing Page - React/TypeScript

La landing page se construye mediante componentes funcionales de React basados en TypeScript, asegurando la tipación estática (type safety) en toda la capa de presentación. Los componentes se nombran en PascalCase (Header, HeroSection, PricingSection) y se centralizan dentro del directorio src/components/. Cada componente declara y recibe propiedades (Props) fuertemente tipadas, mitigando errores comunes en tiempo de compilación.

Para el diseño visual se utiliza Tailwind CSS, aplicando clases utilitarias directamente sobre el atributo className, lo que resulta en un estilo cohesivo, ágil y libre de archivos CSS externos innecesarios. Los colores corporativos de la marca se encuentran centralizados en el archivo de configuración de Tailwind, evitando valores arbitrarios dispersos.

La internacionalización del sitio se gestiona a través de un `LanguageContext` que expone el hook personalizado `useLanguage()`, permitiendo a los componentes acceder y alternar el idioma de manera global sin incurrir en prop drilling. El código mantiene un acceso unificado a las variables de entorno y URLs de redirección mediante constantes globales. Asimismo, componentes clave como `ImageWithFallback` implementan patrones defensivos para mitigar fallos en la carga de recursos multimedia externos, garantizando una experiencia de usuario fluida.



6.2.1.2. Code Quality & Code Security.

Backend - Java/Spring Boot

La calidad en el backend se asegura mediante el desacoplamiento de la arquitectura hexagonal y el uso de DTOs, lo que reduce la complejidad ciclomática y la deuda técnica. En el ámbito de la seguridad a nivel de código, el sistema destaca por:

- Control de Acceso: Integración de Spring Security y tokens JWT. Los endpoints se protegen mediante autorización basada en roles con la anotación `@PreAuthorize`.
- Validación Defensiva: Aplicación estricta de Bean Validation (`@Valid`, `@NotNull`) en controladores para sanitizar los datos de entrada y prevenir ataques de inyección (SQLi) o XSS.

- Transparencia: Todos los flujos de autorización y códigos de respuesta están completamente documentados y declarados en el contrato de OpenAPI/Swagger.

Frontend - Angular/TypeScript

La calidad del frontend se basa en una arquitectura modular por features y en el uso estricto de TypeScript (strict: true), garantizando un tipado fuerte que elimina errores en tiempo de compilación. Las prácticas de seguridad y estabilidad aplicadas incluyen:

- Protección de Navegación: Implementación de guards funcionales (CanActivateFn) que interceptan las rutas y validan la vigencia del JWT y los permisos antes de renderizar cualquier vista.
- Gestión de Estado Inmutable: El uso de Angular Signals previene fugas de memoria (memory leaks) y asegura un flujo de datos reactivo y predecible.
- Saneamiento: Confianza en los mecanismos nativos del framework para sanitizar el DOM y neutralizar vulnerabilidades de Client-Side XSS.

Landing Page - React/TypeScript

A pesar de su naturaleza informativa, la landing page se rige por estándares de calidad basados en componentes funcionales reutilizables y tipados estáticos que evitan caídas de la interfaz. Sus medidas de seguridad y buenas prácticas comprenden:

- Navegación Segura: Uso mandatorio del atributo rel="noopener noreferrer" en enlaces externos para mitigar ataques de manipulación de pestañas (Tabnabbing).
- Gestión de Contexto: Centralización de traducciones mediante LanguageContext fuertemente tipado, evitando referencias nulas en la interfaz.
- Aislamiento: Configuración hermética de variables de entorno para evitar la filtración accidental de URLs o credenciales de desarrollo en el build de producción.

6.2.2. Reviews

6.3. Validation Interviews.

6.3.1. Diseño de Entrevistas.

Segmento #1: Bodegas especializadas por rubro

Presentación del entrevistado:

¿Cuál es tu nombre?

¿Qué edad tienes?

¿Desde cuándo gestionas esta bodega?

¿Cuál es el rubro principal de tu bodega?

El usuario interactúa con la app móvil de la plataforma para gestión de inventarios.

Landing page

Acciones clave dentro del sistema

Navegación interactivo por la interfaz app móvil

User Goal: Registrar

El usuario selecciona la opción "Register", completa los campos solicitados y hace clic en el botón "Registrar".

A continuación, se muestra el panel "Add Card", donde debe llenar los campos relacionados con su tarjeta y correo electrónico.

Una vez que el proceso de pago se complete exitosamente, se notifica al usuario con un mensaje confirmando el vínculo de su tarjeta con la plataforma.

Del mismo modo, si el usuario desea retirar su información o actualizarla, lo podrá hacer a través de su perfil.

Finalmente, hacer clic en el botón "Aceptar".

User Goal: Iniciar sesión

El usuario introduce su correo y contraseña.

Luego hace clic en el botón "Log In".

Después, se le redirige al panel de perfil.

Allí puede editar su información personal y acceder a herramientas según su perfil ("Administrador" o "Empleado").

User Goal: Navegar por el Dashboard

El usuario inicia sesión desde la Landing Page.

Ingresa a la vista principal del Dashboard.

Visualiza el total de productos registrados y la fecha del último proveedor.

Visualiza un resumen de productos próximos a caducar con su respectiva fecha y stock disponible.

Accede a botones de acción rápida como "Historial", "Inventario", "Añadir Productos", "Kits" y "Devolución de productos".

User Goal: Inventario (Producto / Lote)

Ingresa a la sección de Inventario.

Revisa el listado de productos presionando el botón "por producto".

Filtra los productos por categoría, nombre del producto, fecha o stock mínimo.

Consulta el listado con información clave: fecha de entrada, cantidad por unidad, precio, stock mínimo y unidad de medida.

User Goal: Botones Principales (Agregar Producto y Kits)

Pulsa el botón "Agregar Producto".

Rellena los campos solicitados para registrar uno nuevo.

Pulsa el botón "Crear Kit".

Combina productos existentes para crear un kit nuevo.

El usuario pulsa el botón "Añadir Productos" desde el Dashboard.

Visualiza una galería de productos existentes y accede a opciones para editarlos o duplicarlos.

Puede agregar uno nuevo haciendo clic en el botón "+", donde se despliega un formulario con campos como nombre, etiquetas, cantidad, lote, precios, fecha de caducidad y notas.

Desde el menú principal, también accede a la opción "Kits".

En esta sección, selecciona productos existentes y los combina mediante el botón "Seleccionar para kit", indicando cantidad e inventario disponible.

User Goal: Historial de Movimientos

Navega a la sección de Historial.

Visualiza entradas y salidas de productos.

Filtra movimientos por fecha, producto o lote.

El usuario accede a la sección de Historial desde el panel principal.

Filtra los registros por tipo de gestión, categoría, stock promedio y fecha.

Visualiza los movimientos realizados, incluyendo datos como nombre del producto, fecha de consulta, precio unitario, cantidad y total.

Consulta métricas como el stock promedio, estado del producto y stock ideal.

Cuenta con botones para editar o eliminar cada registro y, para los stock promedio, exportar la información y realiza un ticket promedio.

User Goal: Alerta de Stock

El usuario accede a la sección de Alertas desde el menú superior.

Consulta una lista de productos que presentan alertas por stock mínimo o por cercanía a su fecha de vencimiento.

Visualiza detalles como categoría de alerta, tipo de producto, nombre y fecha de alerta.

Puede ingresar a una vista de detalles, generar reportes o confirmar acciones desde botones individuales por producto.

También puede eliminar una alerta específica luego de una confirmación emergente.

Preguntas principales:

- ¿Te resultó fácil encontrar cómo agregar un producto?
- ¿Qué tal fue el proceso para registrar el stock por producto y por lote?
- ¿Te resultó claro el apartado de "Próximos a caducar"? ¿Te pareció útil?

- ¿Probaste la sección de kits? ¿Te resultó útil combinar productos?
- ¿Hubo algo que no entendiste o que te confundió mientras usabas la app?
- ¿Qué te pareció el diseño general de la landing page?
- ¿Tuviste alguna dificultad visual o técnica durante la navegación?

Valoración de experiencia

- Del 1 al 10, ¿qué tan útil te pareció la app para tu bodega?
- ¿Qué funcionalidad te pareció más valiosa?
- ¿Qué función le agregarías sí o sí?
- ¿Recomendarías esta plataforma a otros dueños de bodegas? ¿Por qué?
-

Segmento #2: Startups y emprendedores con necesidades logísticas

Presentación del entrevistado

- ¿Cuál es tu nombre?
- ¿Qué edad tienes?
- ¿Qué tipo de producto vendes o almacenas?
- ¿Tienes local físico, almacén propio o trabajas con terceros?
- Interactúa el usuario con la landing page y la versión web de nuestra plataforma para gestión de stock, registrar compras y generar reportes.

Landing page

Acciones clave dentro del sistema

Navegación interactiva por la interfaz app móvil

User Goal: Registrar

El usuario selecciona la opción "Register", completa los campos solicitados y hace clic en el botón "Registrar".

A continuación, se muestra el panel "Add Card", donde debe llenar los campos relacionados con su tarjeta y correo electrónico.

Una vez que el proceso de pago se complete exitosamente, se notifica al usuario con un mensaje confirmando el vínculo de su tarjeta con la plataforma.

Del mismo modo, si el usuario desea retirar su información o actualizarla, lo podrá hacer a través de su perfil.

Finalmente, hacer clic en el botón "Aceptar".

User Goal: Iniciar sesión

El usuario introduce su correo y contraseña.

Luego hace clic en el botón "Log In".

Después, se le redirige al panel de perfil.

Allí puede editar su información personal y acceder a herramientas según su perfil ("Administrador" o "Empleado").

User Goal: Navegar por el Dashboard

El usuario inicia sesión desde la Landing Page.

Ingresa a la vista principal del Dashboard.

Visualiza el total de productos registrados y la fecha del último proveedor.

Visualiza un resumen de productos próximos a caducar con su respectiva fecha y stock disponible.

Accede a botones de acción rápida como "Historial", "Inventario", "Añadir Productos", "Kits" y "Devolución de productos".

User Goal: Inventario (Lote)

Accede a la sección de Inventario por lote.

Filtra los lotes por nombre del producto, proveedor, fecha de entrada, cantidad o precio.

Consulta el listado con información detallada como proveedor, producto, fecha, cantidad por unidad, precio y unidad de medida.

Gestiona las acciones disponibles para cada lote desde la columna correspondiente.

User Goal: Botones Principales (Agregar Producto y Kits)

Pulsa el botón "Agregar Producto".

Rellena los campos solicitados para registrar uno nuevo.

Pulsa el botón "Crear Kit".

Combina productos existentes para crear un kit nuevo.

El usuario pulsa el botón "Añadir Productos" desde el Dashboard.

Visualiza una galería de productos existentes y accede a opciones para editarlos o duplicarlos.

Puede agregar uno nuevo haciendo clic en el botón "+", donde se despliega un formulario con campos como nombre, etiquetas, cantidad, lote, precios, fecha de caducidad y notas.

Desde el menú principal, también accede a la opción "Kits".

En esta sección, selecciona productos existentes y los combina mediante el botón "Seleccionar para kit", indicando cantidad e inventario disponible.

User Goal: Historial de Movimientos

Navega a la sección de Historial.

Visualiza entradas y salidas de productos.

Filtra movimientos por fecha, producto o lote.

El usuario accede a la sección de Historial desde el panel principal.

Filtra los registros por tipo de gestión, categoría, stock promedio y fecha.

Visualiza los movimientos realizados, incluyendo datos como nombre del producto, fecha de consulta, precio unitario, cantidad y total.

Consulta métricas como el stock promedio, estado del producto y stock ideal.

Cuenta con botones para editar o eliminar cada registro y, para los stock promedio, exportar la información y realiza un ticket promedio.

User Goal: Alerta de Stock

El usuario accede a la sección de Alertas desde el menú superior.

Consulta una lista de productos que presentan alertas por stock mínimo o por cercanía a su fecha de vencimiento.

Visualiza detalles como categoría de alerta, tipo de producto, nombre y fecha de alerta.

Puede ingresar a una vista de detalles, generar reportes o confirmar acciones desde botones individuales por producto.

También puede eliminar una alerta específica luego de una confirmación emergente.

Preguntas principales:

- ¿Qué opinas de las alertas de stock mínimo/máximo y vencimiento? ¿Son suficientes?
- ¿La sección de historial de movimientos te pareció útil para revisar tus registros?
- ¿Te resultó útil la opción de devolución de productos?
- ¿Los reportes del historial son útiles para tomar decisiones?
- ¿Hubo algo que no entendiste o que te confundió mientras usabas la app?
- ¿Qué te pareció el diseño general de la landing page?
- ¿Tuviste alguna dificultad visual o técnica durante la navegación?

Valoración de experiencia

- Del 1 al 10, ¿qué tan útil te pareció la app móvil para tu bodega?
- ¿Qué funcionalidad te pareció más valiosa?
- ¿Qué función le agregarías sí o sí?
- ¿Recomendarías esta plataforma a otros emprendedores? ¿Por qué?

6.3.2. Registro de Entrevistas.

6.3.3. Evaluaciones segun heurísticas.

6.4. Auditoria de Experiencias de Usuario.

6.4.1. Auditoria realizada.

6.4.1.1. Información del grupo auditado.

6.4.1.2. Cronograma de auditoria realizada.

6.4.1.3. Contenido de auditoría realizada.

6.4.2. Auditoria recibida.

6.4.2.1. Información del grupo auditor.

6.4.2.2. Cronograma de auditoría recibida.

6.4.2.3. Contenido de auditoría recibida.

6.4.2.4. Resumen de modificaciones para subsanar hallazgos.

Capitulo VII: DevOps Practices

7.1. Continuous Integration

7.1.1. Tools and Practices

7.1.2. Build & Test Suite Pipeline Components

7.2. Continuous Delivery

7.2.1. Tools and Practices

7.2.2. Stages Deployment Pipeline Components

7.3. Continuous deployment

7.3.1. Tools and Practices

7.3.2. Production Deployment Pipeline Components

7.4. Continuous Monitoring

7.4.1. Tools and Practices

7.4.2. Monitoring Pipeline Components

7.4.3. Alerting Pipeline Components

7.4.4. Notification Pipeline Components

Part III: Experiment-Driven Lifecycle

Capitulo VIII: Experiment-Driven Development

8.1. Experiment Planning

8.1.1. As-Is Summary

8.1.2. Raw Material: Assumptions, Knowledge Gaps, Ideas, Claims

8.1.3. Experiment-Ready Questions

8.1.4. Question Backlog

8.1.5. Experiment Cards

8.2. Experiment Design

8.2.1. Hypotheses

8.2.2. Domain Business Metrics

8.2.3. Measures

8.2.4. Conditions

8.2.5. Scale Calculations and Decisions

8.2.6. Methods Selection

8.2.7. Data Analytics: Goals, KPIs and Metrics Selection

8.2.8. Web and Mobile Tracking Plan

8.3. Experimentation

8.3.1. To-Be User Stories

8.3.2. To-Be Product Backlog

8.3.3. Pipeline-supported, Experiment-Driven To-Be Software Platform Lifecycle

8.3.3.1. To-Be Sprint Backlogs

8.3.3.2. Implemented To-Be Landing Page Evidence

5.2.3. Implemented Frontend-Web Application Evidence

8.3.3.4. Implemented To-Be Native-Mobile Application Evidence

8.3.3.5. Implemented To-Be RESTful API and/or Serverless Backend Evidence

8.3.3.6. Team Collaboration Insights

8.3.4. To-Be Validation Interviews

8.3.4.1. Diseno de Entrevistas

8.3.4.2. Registro de Entrevistas

8.4. Experiment Aftermath & Analysis

8.4.1. Analysis and Interpretation of Results

8.4.2. Re-scored and Re-prioritized Question Backlog

8.5. Continuous Learning

8.5.1. Shareback Session Artifacts: Learning Workflow

8.6. To-Be Software Platform Pre-launch

8.6.1. About-the-Product Intro Video

Conclusiones

Conclusiones y recomendaciones

Bibliografia

Anexos
